CSC 126 Computer Vision Midterm Project Rubrics

Objective: Design and implement a working computer vision application that integrates classical OpenCV techniques to solve a simple real-world problem (e.g., attendance/face monitoring, traffic or people counting, shape or object sorting, drowsiness detection, motion-triggered security). Present and demonstrate it live. (100 pts)

Expected output: Students are required to submit the following:

1. **Live demo presentation**
   15 min demo + 5 min Q&A.
2. **Short documentation**
   a 2-4 page PDF that states your project's title and objectives, describes the real-world problem it solves, and briefly explains the OpenCV techniques you used for topics a-f. It also includes captioned screenshots of your results, a discussion of accuracy, behavior, and limitations, and a conclusion with key learnings and possible improvements.
3. **Python source code**
   Submit the full repository with well commented .py or .ipynb files.

Instructions:

1. This activity can be done individually or by group (**max of 3** members only). You will create a computer vision application using **Python and OpenCV** that applies core image-processing and analysis techniques. Your project should solve a simple real-world problem or demonstrate a meaningful computer vision task (e.g., object tracking, shape detection, motion detection, face detection, filtering, etc.).
2. Your project must include at least three (3) of the following topics from the Laboratory: Foundation in Classical Computer Vision with OpenCV:
   - **a. Image Operations**
   - **b. Image Segmentation**
   - **c. Haar Cascade Classifier**
   - **d. Image Analysis and Transformation**
   - **e. Motion and Object Tracking**
   - **f. Facial Landmark Detection**
3. Your application must process either:
   - A **static image**, or
   - A **recorded video**, or
   - A **live webcam feed**
4. Output must show:
   - **Processed frames**
   - **Detected objects/shapes/features**
   - **Key results** clearly visualized (e.g., bounding boxes, contours, overlays)
   - **Bounding boxes, labels, overlays, masks**
5. Prepare a short documentation (2–4 pages) describing:
   - Title and Objectives of the Project
   - Description of the Problem or Use-Case
   - Techniques Used (with brief explanations)
   - Screenshots of your results/output
   - Discussion of accuracy, behavior, or limitations
   - Conclusion
6. Present a live demo (15 minutes, plus 5 minutes Q&A) showing:
   - Your Python application running (as a script, desktop app, or web interface)
   - The application processing images/video or a live webcam feed
   - The detection or analysis results for your chosen topics

- A short explanation of how your project works

7. Submit the link to your GitHub repository containing:
   - Your complete, well-commented Python source code (import statements, preprocessing steps, detection/analysis code, visualization code, and clear function or loop structure)
   - A README.md with the project description, setup instructions, and how to run the application
   - A requirements.txt listing the dependencies
   - Sample input files and screenshots of the output

	Excellent 20	Satisfactory 15	Needs Improvement 10	No Submission (0)
Clarity of documentation and Code quality	The documentation is clear, complete, and well-organized, presenting the project's objectives, methods, screenshots, and results with strong explanations. Writing is easy to follow and free of major errors. The code is clean, well-structured, and properly commented, using meaningful variable names and logical organization. Both documentation and code demonstrate a high level of understanding and professionalism.	The documentation is understandable but may lack detail or organization in some areas. Explanations or screenshots may be brief or incomplete. The code works but may contain minor formatting issues, limited comments, or inconsistent structure. Overall, the submission shows competence but requires improvement in clarity and organization.	The documentation is unclear, incomplete, or poorly structured, with missing explanations or unlabeled screenshots. The code is difficult to read, lacks comments, or is poorly organized, making the logic hard to follow. Significant improvements are needed in both writing quality and coding practices.	No documentation or code is submitted, or the submitted files contain no meaningful or functional content.
Live demonstration	The live demo is clear, well-organized, and easy to follow. It shows the full workflow, including the application running, the program output, and a brief explanation of how the system works. The verbal explanations are understandable, the presenters handle the application smoothly, and the demo stays within the time limit.	The live demo covers most parts of the workflow but may lack clarity in some sections. Some explanations may be brief or not well detailed. There may be minor issues such as unclear transitions, slight disorganization, small technical hiccups, or small deviations from the time limit. Overall, the demo is still understandable.	The live demo is difficult to follow or incomplete. Key steps of the workflow are missing or not explained. The demo may be unclear, unorganized, over time, or affected by technical issues that hurt understanding. It does not effectively show how the system works.	No presentation.

	Excellent 30	Satisfactory 20	Needs Improvement 10	No Submission (0)
Correct use of OpenCV techniques	The project demonstrates accurate and effective use of at least three OpenCV techniques. All implemented methods (e.g., preprocessing, edge detection, contour extraction, feature detection, object tracking, etc.) are correctly applied and clearly relevant to the project's goal. The techniques are used appropriately, produce correct results, and contribute meaningfully to the overall functionality of the application.	The project includes several OpenCV techniques, but some may be partially implemented, contain minor errors, or lack clear justification. Most functions work, but certain steps or outputs may not be fully optimized or correctly applied. The techniques still contribute to the project but show room for improvement in accuracy or completeness.	Only one or two OpenCV techniques are correctly used, or the methods applied show significant misunderstanding. Several functions may be incorrectly implemented, irrelevant to the project, or produce incorrect or unclear output. The use of OpenCV is minimal, incomplete, or poorly executed.	No OpenCV techniques are demonstrated, or the student did not submit any working implementation.
Functionality of the application	The application runs smoothly and performs all intended computer vision tasks correctly. All features function as expected with accurate and consistent results. The program successfully processes images, videos, or webcam input without errors, crashes, or major delays. Outputs (e.g., detections, contours, overlays) are clear, reliable, and aligned with the project's goals.	The application works but may contain minor issues, such as occasional errors, inconsistent detections, or minor performance delays. Most features run correctly, and the program still achieves the main task. Output is understandable but may lack consistency or refinement.	The application runs but has significant problems in functionality. Major errors, incorrect outputs, unstable performance, or incomplete features are present. The program may run only partially, crash frequently, or fail to deliver the expected computer vision results. Functionality is limited or not aligned with the project objectives.	No functional application is provided, or the code does not run at all.

Prepared By:

LYNDON KING L. OBENZA

Faculty

JOSHUA T. TUBO

Faculty

PRINCESS BEVERLY R. ALA-AN

Faculty

Approved By:

ROLAND P. ABAO, Ph.D

Chairperson, CS Department

JAYMER M. JAYOMA, DIT

Dean, College of Computing and Information Sciences

JENELYN R. ASIS, Msc.

IT Chairperson